Document de processus technique - pili-pos SCE 2.0

Distributeur : Bruno Moyse / nuagemagique.dev (Bruxelles, BE) **Nom du produit :** pili-pos **Version :** 1.0.0 (build de soumission cert) **Clients cibles :** établissements HoReCa en Belgique (restaurants avec service en salle) **Date de soumission :** 2026-10-03 **Specs implémentées :** POS v2.2 + FDM v1.4 + Addendum 1 + Basic Set v1.8 + Reports v1.3 + Advanced Set v1.6

1. Vue d'ensemble du système

`pili-pos` est une **application desktop Tauri v2** s'exécutant sur Linux x86_64 (Debian 13 minimal + compositeur Cage Wayland en mode kiosk). Elle cible les terminaux POS Penetek A3216 M82 / Advantech USC-360J, se connecte à un FDM certifié SPF Finances par LAN, pilote une imprimante thermique ESC/POS Epson TM-m30III, et persiste chaque événement fiscal dans une base SQLite locale append-only.

```
┌──────────────────────────────────────────────────────────────────┐
│                       Terminal POS                                │
│                                                                  │
│  ┌────────────────────────────────────────────────────────────┐  │
│  │  UI Vue 3 + Pinia (touch-optimisée, Composition API)       │  │ │
│  └────────────────────────────┬───────────────────────────────┘  │
│                               │ tauri::invoke (IPC)               │
│  ┌────────────────────────────▼───────────────────────────────┐  │
│  │  Couche commandes Rust (crates/pos-engine/src/commands)         │ │
│  │  checkout / order / refund / report / tables / social /...  │ │
│  └──┬─────────────────┬──────────────────┬────────────────────┘  │
│     │                 │                  │                        │
│  ┌──▼───────┐  ┌────────▼─────┐  ┌─────────▼───┐                 │
│  │pos-core  │  │ pos-fiscal   │  │  pos-print  │                 │
│  │ types &  │  │ types évt    │  │ ESC/POS     │                 │
│  │ règles   │  │ client FDM   │  │ tickets     │                 │
│  │          │  │ signatures   │  │ cuisine     │                 │
│  └──────────┘  │ compteurs    │  │ tiroir      │                 │
│                │ archive      │  └─────────────┘                 │
│                └──────┬───────┘                                   │
│                       │                                           │
│                 ┌─────▼────────┐  ┌──────────────┐                │
│                 │   pos-db     │  │  pos-peppol  │                │
│                 │ SQLite +     │  │  UBL 2.1     │                │
│                 │ migrations   │  │  AccessPt    │                │
│                 │ versionnées│  └──────────────┘                │
│                 └──────────────┘                                  │
└──────────────────────────────┬───────────────────────────────────┘
                               │ HTTP GraphQL (LAN, mTLS optionnel)
                               ▼
┌──────────────────────────────────────────────────────────────────┐
│  FDM module2 (simulateur en ligne pour la machine de test)        │
│  - Signature ECDSA secp256r1 (clé détenue par le FDM)              │
│  - Stratégie compteurs : par type d'événement                      │
│  - Uplink cloud vers SPF Finances (hors périmètre POS)             │
└──────────────────────────────────────────────────────────────────┘
```

Structure du code. Le noyau fiscal est le moteur `crates/pos-engine`, sans dépendance à Tauri : les fonctions de commande (`crates/pos-engine/src/commands`), l'état partagé (`state.rs`), l'ouverture du moteur (`engine.rs` : base SQLite, migrations, reprise après arrêt brutal, chargement de la configuration, verrou exclusif sur le fichier `engine.lock` du répertoire de données) et les tâches de fond (`background.rs` : Z automatique, file d’envoi des e-mails, scrutation et file d’envoi HubRise). Le binaire `pili-pos` (`src-tauri`) est une enveloppe mince : une enveloppe IPC par commande (`src-tauri/src/ipc`, un seul appel vers le moteur), la fenêtre, le contrôle de santé sur `127.0.0.1:9999` et la résolution de la configuration (variable `PILI_POS_CONFIG`, sinon `/etc/pili-pos/pos.json`). Le `posSwVersion` des événements et des tickets est une valeur que l’enveloppe transmet au démarrage ; sur Linux elle reste `pili-pos/` suivi de la version du workspace. Le harnais de tests `crates/pos-engine-testkit` ne fait pas partie du binaire certifié.

2. Comportement par événement

Événement	Déclencheur	Chemin de code POS	Notes
N (Vente normale)	L’opérateur tape “Payer” (sur place `dine_in` ou à emporter `takeaway`)	`ipc/checkout::checkout` → `commands/checkout::do_checkout` / `do_checkout_split` → `sell` → `signing::sign` : archive + clôture dans une seule tx SQLite	Blocage-sur-déconnexion-FDM imposé
P (Pro forma)	L’opérateur envoie la commande en cuisine	`commands/tables::do_sign_table_order`	Table ouverte ; pas d’incrément du compteur N
P (Changement de table, `COST_CENTER_CHANGE`)	L’opérateur change une commande de table	`commands/table_transfer::do_transfer_table`	Enregistre d’abord les articles non envoyés (P), puis signe le déplacement ; la table change dans la transaction d’archivage ; pas d’incrément du compteur N
T (Formation)	L’opérateur est en mode formation	Comme N mais `is_training=true` dans l’input ; watermark UI	
C (Copie)	Réimpression d’un ticket antérieur	`commands/copy::do_copy_ticket_with`	Porte un `fdmRef` vers l’original
I (Facture)	L’opérateur établit une facture à partir de ventes déjà encaissées (Rapports, onglet Tickets, « Facture »)	`commands/invoice::do_sign_invoice_fiscal`	Une vente, une facture ; numéro `<année><séquence>` sans trou ; copie C possible ; `FACTURATION_PEPPOL_FR.md`
F (Financier)	Mouvement caisse, ouverture seule du tiroir	`commands/financial::do_money_in_out` / `do_open_drawer_fiscal`	F-WAR pour ouverture tiroir
F (Correction de paiement, `PAYMENT_CORRECTION`)	Le Gérant corrige le moyen d’une ligne de paiement d’une vente (onglet Tickets)	`commands/payment_correction::do_correct_payment`	PIN du Gérant ; `fdmRef` de la vente ; une correction par vente ; aucun ticket, tiroir fermé ; rapports X/Z : corrections positives et négatives

Événement	Déclencheur	Chemin de code POS	Notes
S (Social)	Pointage entrée / sortie de l'opérateur ; pointage d'un autre employé par le Gérant ; pointage par badge (option « Lecteur de badges »)	`commands/social::do_work_in` / `do_work_out` ; `do_clock_in_employee` / `do_clock_out_employee` ; `commands/badges::do_badge_tap` (via `commands/social::sign_work_event`)	INSZ + n° employeur
R (Rapport)	Rapports Z et X	`commands/report::do_generate_x_report` / `do_generate_z_report`	Z est Turnover-Z + User-Z atomique

Pointage par badge (option, désactivée par défaut). Le lecteur USB (clavier émulé : numéro puis Entrée) est lu par l'interface, qui ne fait que reconnaître la rafale de touches et transmettre le numéro brut à `commands/badges::do_badge_tap`. Le moteur seul décide : numéro normalisé, empreinte HMAC-SHA256 avec la clé de l'appareil (`badge_key.rs`, fichier `badge.key` hors de la base), anti-rebond de 10 secondes, recherche dans `staff_badges`, puis bascule de présence par `commands/social::sign_work_event` (événement S au nom du titulaire du badge, identique à M140 et M141), ouverture de session sans PIN pour un Serveur ou un Apprenti si la connexion par badge est activée, PIN toujours exigé d'un Gérant. Le lien (`do_link_badge`, Gérant + son PIN), le retrait (`do_revoke_badge`) et les réglages (`do_set_badge_settings`) sont réservés au Gérant et journalisés. Voir `FLUX_DONNEES_FR.md` §6 ter et `REPONSES_MANUEL_CERT_FR.md` §III.E.

Chaque branche :

1. Charge les lignes pertinentes depuis SQLite (les requêtes sont dans `pos-db/src/queries/`).
2. Construit la struct d'input SCE-2.0 (`pos-fiscal::inputs::*`).
3. Sous le verrou de signature unique (`commands/signing.rs`), lit le prochain numéro de ticket, choisit la période d'ouverture, calcule la clé de doublon (voir §3.4) et le corps JSON canonique (voir §3.3), puis stocke la requête exacte dans `pending_signs`.
4. Appelle la méthode async `FdmClient::sign_*` correspondante (une requête restée sans réponse est d'abord renvoyée à l'identique, voir §4).
5. Sur succès : avance le compteur, ajoute à `fiscal_events`, ferme ou marque la commande et clôt la ligne `pending_signs`, dans une seule transaction SQLite.
6. Retourne les données de ticket à l'UI ; l'UI déclenche l'impression via `pos-print`.

Remise de caisse. Les commandes `apply_discount` et `remove_discount` (Gérant confirmé par son PIN) calculent la remise sur le net de la commande, la répartissent au centime près sur les lignes et la transmettent en `priceChanges` : sur une commande en cours elle part avec la vente (N) ; sur une commande déjà enregistrée en salle elle signe un événement P immédiat, repris par `pending_signs` si le FDM ne répond pas. Le motif et le Gérant restent dans `till_discounts` (migration 30).

Note de cuisine. La commande `set_line_note` (tout le personnel) vérifie la note (60 caractères au plus, imprimables en CP858 : `pos_print::escpos::is_cp858_printable`), la range dans la colonne locale `order_lines.kitchen_note` (migration 31) et, si la ligne est déjà partie en cuisine et que la note diffère de celle déjà imprimée, imprime un bon « MODIFICATION » (`crates/pos-print/src/bar_ticket.rs::format_note_slip`). Aucune signature, aucun événement : la note reste locale et les constructeurs des messages FDM ne la lisent pas.

3. Signature / vérification

3.1 Transport

- **Protocole :** GraphQL sur HTTP/HTTPS.
- **Endpoint :** configuré via `pos.json` (`fdm_url`).

- **Sondage de statut :** le moteur interroge `FdmClient::status` (`crates/pos-engine/src/commands/fdm_monitor.r` boucle de `background.rs`) dès qu'une session s'ouvre, puis toutes les 30 s tant que le FDM répond, avec un recul jusqu'à 5 min tant qu'il ne répond pas ; l'interface ne fait que recevoir le résultat (événement `fdm-status`). Le checkout n'appelle pas le statut : chaque vente part directement vers `signing::sign` (§5).

3.2 Signature de réponse

- Algorithme : **ECDSA secp256r1**, calculé par le FDM avec sa propre clé. Le POS ne détient pas la clé publique du FDM et ne vérifie pas la signature ECDSA ; il la conserve telle quelle, avec la réponse complète, dans `fiscal_events.fdm_response_json`.
- Contrôle de cohérence avant archivage : `pos_fiscal::response_check::check_sign_result`, appelé par `FdmClient` pour chaque réponse de signature (y compris les deux moitiés du Z M184). Il exige l'écho de `posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId` et `terminalId`, l'`eventOperation` et l'`eventLabel` de la mutation (T en formation), une `digitalSignature` en base64 valide et, pour N, une `shortSignature` égale à Hex(SHA1(Base64Decode(digitalSignature))) (`pos_fiscal::signature::compute_short_signature`) ainsi qu'une `verificationUrl`. Toute divergence produit l'erreur `FDM_RESPONSE_MISMATCH` : rien n'est archivé ni imprimé.
- La short signature apparaît sur le ticket imprimé (40 caractères hexadécimaux).

3.3 JSON canonique

- `pos-fiscal::canonical::to_canonical()` sérialise une valeur en JSON canonique style RFC 8785 : clés triées par codepoint Unicode à chaque niveau, pas d'espace insignifiant, nombres normalisés (pas de zéros traînants, `-0` → `0`).
- Utilisé pour **chaque** corps de requête envoyé au FDM, de sorte qu'un flux de signature reproductible par hash est possible.
- Test de propriété dans `crates/pos-fiscal/src/canonical.rs::tests` (vérification de déterminisme sur 50 itérations).

3.4 Clé de doublon FDM

- `pos-fiscal::inputs::IdempotencyKey { pos_id, pos_date_time, terminal_id, event_label, pos_fiscal_ticket_no }`.
- Chaque type d'input implémente `.idempotency_key()`.
- Règle de doublon du FDM : même clé et contenu identique, le FDM renvoie sa réponse déjà signée avec l'avertissement `DUPLICATE_REQUEST` ; même clé et contenu différent, il répond `INVALID_REQUEST`. Le POS ne reconstruit donc jamais une requête pour un numéro déjà envoyé : il renvoie la requête stockée, octet pour octet (§4).
- Les tests unitaires dans `crates/pos-fiscal/src/inputs.rs` couvrent l'égalité de la clé et le mapping de label par type.

4. Stratégie de compteurs

Voir `STRATEGIE_COMPTEURS_FR.md`. **Compteurs par type d'événement** stockés dans la table `fiscal_counters` ; le numéro est lu sous le verrou de signature et le compteur n'avance que dans la transaction SQLite qui archive l'événement signé, de sorte que :

- Les compteurs sont strictement monotones par type d'événement et deux opérations ne partagent jamais un numéro (verrou unique, `signing.rs`).
- Une requête non envoyée ou refusée par le FDM ne consomme pas son numéro - pas de trou.
- Une requête dont la réponse est perdue reste ouverte dans `pending_signs` ; elle est renvoyée à l'identique (même numéro, même contenu) avant toute autre signature, et le FDM renvoie sa réponse avec `DUPLICATE_REQUEST`. L'événement est archivé une fois, sous son premier numéro.
- Tant qu'elle n'est pas résolue, les autres signatures sont refusées (`FDM_ANSWER_PENDING`) ; la tâche de fond (60 s) renvoie la requête.

- Risques résiduels : un FDM qui aurait perdu sa mémoire des doublons signerait le renvoi comme un nouvel événement (détecté par les compteurs et journalisé, non évitable par le POS) ; une action de l'écran peut attendre jusqu'à 30 s derrière un Z automatique ou un renvoi (`signing_busy` ensuite).

5. Comportement hors-ligne / déconnexion

- **Pas de FDM, pas d'événement.** Chaque commande fiscale (vente, remboursement, commande, addition, copie, pointage, mouvement de caisse, ouverture de tiroir, rapport) fait signer son événement par le FDM *avant* d'écrire quoi que ce soit : il n'y a pas de pré-contrôle de statut, la vente passe directement par `signing::sign`. Si le FDM n'est pas joignable, `FdmError::outcome()` donne `NotSent` (la ligne `pending_signs` passe à `not_sent`) et la commande renvoie une erreur (`success: false`, « FDM not connected - cannot complete transaction » pour une vente, transmise à l'interface), rien n'est archivé, aucun compteur n'avance, la commande reste ouverte et le tiroir reste fermé. L'opérateur réessaie une fois le FDM revenu. Avec un terminal de paiement relié, le même principe s'applique en amont : la carte n'est demandée au terminal que si le FDM est opérationnel à cet instant (§6quater). Si la requête avait été envoyée et que la réponse est perdue, la requête est conservée et renvoyée à l'identique avant toute autre signature (§4) : l'événement n'est ni perdu ni doublé.
- **Aucun mode hors-ligne**, dans aucune build : le code ne contient aucun chemin qui archive ou imprime une vente non signée.
- L'interface affiche en permanence l'état du FDM (voyants `StatusLights.vue`, alertes `FdmAlerts.vue`), interrogé par le moteur (`commands/fdm_monitor.rs`) toutes les 30 s avec recul jusqu'à 5 min, avec un message lisible pour chaque code d'erreur du FDM. Sans réponse du FDM, l'état passe à `connected = false`, gravité « bloquante ».
- Seule exception : les entrées de commande (P) non encore envoyées restent en attente dans `order_entries` et partent à l'enregistrement suivant (commande, addition ou vente) ; ce ne sont pas des événements tant que le FDM ne les a pas signées.

6. Pipeline d'impression

- Crate : `crates/pos-print`.
- Transport : `pos-print::transport` n'implémente que TCP brut (port 9100) vers la TM-m30III à une IP LAN statique. Pas de transport USB ni série.
- Layout : `pos-print::receipt` formate le ticket fiscal SCE-2.0 selon la spec POS v2.2 §III.3 - en-tête (nom restaurant, adresse, TVA), corps (lignes avec label TVA), table de répartition TVA, bloc de données de contrôle issu de la réponse FDM, QR Code (Model 2, ECC M, URL 38 caractères ; version 3, 29×29 modules - voir `TICKET_NUMERIQUE_FR.md` §11).
- Tickets de commande P (bar / cuisine) : layout séparé (`crates/pos-print/src/bar_ticket.rs::format_bar_tick` envoyé par `commands/bar_slip.rs`), qui imprime aussi la note de cuisine de la ligne sous l'article. Le ticket cuisine `kitchen_ticket::format_kitchen_ticket` ne sert qu'aux commandes en ligne.
- Texte externe (noms de produits, notes, adresses) : passé par `escpos::printable`, qui supprime les octets de contrôle (un ESC ou GS serait une commande d'imprimante) et translittère les caractères hors CP858.
- Ouverture tiroir : séquence ESC/POS `0x1B 0x70 0x00 0x19 0xFA` émise dès qu'un paiement `cash` clôture une vente.

6bis. Pipeline d'envoi du ticket numérique (e-mail)

- Crate : `crates/pos-mail`, plus les tables `ticket_deliveries` et `original_prints` de `crates/pos-db` (migrations `005`/`006`).
- Rendu (`pos-mail::render`) : corps texte = `pos_print::render` appliqué au **même** `format_receipt` que le papier ; corps HTML + QR PNG en pièce jointe uniquement quand le FDM a renvoyé une URL de vérification (événements N).
- Transport (`pos-mail::tem`) : client HTTP vers l'API Scaleway Transactional Email (`X-Auth-Token`, région `fr-par` par défaut).

- File d'attente (`pos-mail::outbox`) : tentative immédiate bornée à 8 s à l'encaissement, puis réessais en tâche de fond avec recul exponentiel (`commands::delivery::run_outbox_once`, toutes les 30 s). Détail complet : `TICKET_NUMERIQUE_FR.md`.

6ter. Commandes en ligne

- Crates : `crates/pos-hubrise` (réception, file d'envoi, menu reflet), `crates/pos-catalog` (validation et application du catalogue), commandes du moteur `crates/pos-engine/src/commands/webshop.rs` (enveloppes IPC : `src-tauri/src/ipc/webshop.rs`).
- Interrupteur par installation : `hubrise_enabled` dans `pos.json`, puis un Gérant dans Réglages ; journal en ajout seul `integration_switch_log` (migration 25). Coupé : aucune boucle ne contacte HubRise et aucune commande n'est acceptée ; une commande déjà signée reste soldable.
- Chaque commande en ligne se termine par un N signé sur ce terminal, au nom de l'employé qui l'accepte (P puis N si elle est payée à la remise). Détail : `COMMANDES_EN_LIGNE_FR.md`.

6quater. Terminal de paiement par carte relié (facultatif)

- Crates : `crates/pos-eft` (contrat, trois pilotes, terminal de test), commandes du moteur `crates/pos-engine/src/commands/eft.rs` (enveloppes IPC : `src-tauri/src/ipc/eft.rs`), table `eft_payments` (`crates/pos-db/src/schema_eft.rs`, migration 27). Présent dans la build certifiée de toute installation, mais inactif sans section `payment_terminal` dans `pos.json` : la carte se saisit alors à la main (ligne de paiement `MANUAL`, `CARD_DEBIT`) et rien ne change.
- Trois protocoles, tous d'après une spécification publique : Worldline Terminal API (nexo Retailer 5.1, JSON sur HTTPS, par le nuage Worldline), CCV Payment API (méthode `terminal`, JSON sur HTTPS, par le nuage CCV) et OPI (XML sur TCP, sur le LAN du restaurant). Le terminal Worldline C-TEP (Yomani, Yoximo) n'est pas pris en charge : protocole fermé, bibliothèque et certification Worldline requises. Chaque fournisseur certifie son intégration avant tout usage réel.
- Étapes d'un paiement (`do_eft_pay`) : contrôle de la demande, **FDM opérationnel à cet instant** (rien n'est débité pour une vente qui ne pourrait pas être signée, AM art. 14), **ligne `eft_payments` écrite avant l'appel du terminal**, appel (la référence UUID est la clé d'idempotence du fournisseur), enregistrement de la réponse. Une réponse inconnue (coupure, délai dépassé) se redemande avec `do_eft_check` ; un paiement resté `sent` lors d'un arrêt brutal devient `unknown` au démarrage et se vérifie de la même façon.
- Règles de la vente : un paiement qui attend sa réponse bloque la vente de sa commande (`eft_pending`) ; un paiement accepté doit être repris par la vente pour son montant exact, sinon la vente est refusée (`eft_payment_not_usable`) ; le paiement est marqué utilisé dans la transaction qui archive l'événement N.
- Ligne de paiement signée (POS spec v2.2, `paymentLine.inputMethod` : « le terminal de paiement renvoie lui-même le type de carte utilisé ») : `inputMethod AUTOMATIC`, type fourni par le terminal (`CARD_DEBIT` id "2" "Card", `CARD_CREDIT` id "6" "Credit card", `CARD_UNKNOWN` id "7" "Card (type unknown)"), `provider` = marque de la carte, `reference` = identifiant de transaction du terminal (`crates/pos-fiscal/src/payment.rs`). Une carte de chèques-repas (marque Edenred, Pluxee, Monizze ou Payflip, reconnue par la marque normalisée que le terminal rend, correspondance exacte) est signée `CHEQUE_MEAL` id "3" "Meal voucher", `AUTOMATIC`, avec la marque en `provider` : la même identité que le chèque-repas saisi, donc une seule rangée dans les rapports. Les variantes éco, cadeau, sport et culture ne sont pas des chèques-repas. Les paiements carte ne sont jamais arrondis, pour toute installation. Le remboursement reste rendu en espèces ; l'annulation d'un paiement carte accepté n'est pas prise en charge.
- Le mode formation ne débite jamais une vraie carte (`eft_training`).

7. Persistance

- Moteur : **SQLite** (DB en un seul fichier dans `~/.local/share/pili-pos/pos.db` en dev, `/var/lib/pili-pos/pos.db` en prod).
- Mode : WAL (`PRAGMA journal_mode=WAL`).

- Migrations : versionnées via `pos-db::migrate::run_migrations`. La table `_migrations_applied` suit `(version, name, applied_at)`. Le chemin de rétro-compatibilité pour la migration one-shot d'origine est dans `migrate::backfill_legacy_db`.
- Plan de salle : les numéros de table en usage sont stockés dans `table_plan` (valeur de départ : clé `tables` de `pos.json`, ou 1 à `table_count`), puis modifiables par un Gérant ; chaque changement est inscrit dans `table_plan_log` (ajout seul, migration 11). Le plan ne touche à aucune donnée fiscale : il décide seulement des tables que l'écran des tables affiche, plus toute table hors plan qui a encore une commande ouverte.
- Paiements sur le terminal relié : table `eft_payments` (migration 27, ajout seul : jamais supprimée, une réponse définitive ne change plus ; voir `SCHEMA_BD_FR.md` et `SECURITE_BD_FR.md`).
- Log fiscal append-only : table `fiscal_events`. Le schéma impose la contrainte CHECK sur `event_type` et des triggers SQLite refusent tout `DELETE` et toute réécriture (`fiscal_events_no_delete`, `fiscal_events_no_rewrite`) ; le code applicatif n'en émet d'ailleurs jamais. Voir `SECURITE_BD_FR.md`.
- Sauvegarde : après chaque rapport Z, juste après son archivage (`signing::follow_up` pour `ReportZ`, qui appelle `commands::report::backup_database_impl`), snapshot via l'API SQLite Online Backup. Un échec de sauvegarde est journalisé et n'annule pas le Z. Une fois la copie vérifiée (`PRAGMA quick_check`), le moteur ne garde que ses 7 sauvegardes les plus récentes (`report::BACKUPS_KEPT`) ; `pos.db` garde tout le journal fiscal.

8. Configurations de build

Build	Features Cargo	Produit
Dev	(par défaut) `hubrise`	App complète
Cert	`--no-default-features --features cert` (`hubrise` + interface embarquée)	Binaire certifié : commandes en ligne présentes, activées par installation (`COMMANDES_EN_LIGNE_FR.md`) ; facturation B2B (événement I) et file d'envoi Peppol présentes dans tous les binaires, sans fonction Cargo (`FACTURATION_PEPPOL_FR.md`)

Vérification sur le binaire cert (`scripts/build_and_hash.sh`) :

```
$ strings target/release/pili-pos | grep -c integration_switch_log  # commandes en ligne présentes
$ nm target/release/pili-pos | grep -c pos_peppol             # code de facturation présent (compte i
```

9. Carte des sources

Préoccupation	Crate / fichier
Types d'événements fiscaux & structs d'input	`crates/pos-fiscal/src/inputs.rs`
Client GraphQL FDM	`crates/pos-fiscal/src/fdm_client.rs`
JSON canonique	`crates/pos-fiscal/src/canonical.rs`
Short signature (SHA-1)	`crates/pos-fiscal/src/signature.rs`
Contrôle des réponses FDM	`crates/pos-fiscal/src/response_check.rs`
Compteurs fiscaux	`crates/pos-fiscal/src/counter.rs`
Archivage tickets	`crates/pos-fiscal/src/archive.rs`
Schéma base de données	`crates/pos-db/src/schema.rs`
Runner de migrations	`crates/pos-db/src/migrate.rs`
CRUD commandes	`crates/pos-db/src/queries/orders.rs`
Helpers requêtes rapport Z	`crates/pos-db/src/queries/reports.rs`

Préoccupation	Crate / fichier
Moteur fiscal (ouverture, verrou, tâches de fond)	`crates/pos-engine/src/{engine,background,state}.rs`
Enveloppe Tauri (fenêtre, contrôle de santé, configuration)	`src-tauri/src/main.rs`
Enveloppes IPC des commandes	`src-tauri/src/ipc/`
Harnais de tests (hors binaire certifié)	`crates/pos-engine-testkit/`
Flux checkout	`crates/pos-engine/src/commands/checkout.rs`
Tables / split / merge	`crates/pos-engine/src/commands/tables.rs`
Plan de salle (lecture, changement par un Gérant)	`crates/pos-engine/src/commands/table_plan.rs`
Remboursement + note de crédit	`crates/pos-engine/src/commands/refund.rs`
Rapports (X/Z)	`crates/pos-engine/src/commands/report.rs`
Social (événements S)	`crates/pos-engine/src/commands/social.rs`
Financier (événements F)	`crates/pos-engine/src/commands/financial.rs`
Copie (événements C)	`crates/pos-engine/src/commands/copy.rs`
Paiement par terminal relié (`do_eft_pay`, `do_eft_check`, `do_eft_abort`)	`crates/pos-engine/src/commands/eft.rs`
Pilotes du terminal (Worldline, CCV Payment API, OPI)	`crates/pos-eft/src/`
Table `eft_payments` (migration 27)	`crates/pos-db/src/schema_eft.rs`
Ligne de paiement carte (types, `AUTOMATIC`)	`crates/pos-fiscal/src/payment.rs`
Sérialiseur UBL Peppol	`crates/pos-peppol/src/ubl.rs`
Facture imprimée (ESC/POS)	`crates/pos-print/src/invoice.rs`
File d’envoi Peppol (essais, journal)	`crates/pos-engine/src/commands/peppol_queue.rs`
Produit épuisé	`crates/pos-engine/src/commands/sold_out.rs`
Éditeur de carte	`crates/pos-engine/src/commands/menu_admin.rs`
Rapport de ventes non fiscal	`crates/pos-engine/src/commands/sales_report.rs`
Tables de la v1.0.0 : facturation, épuisé, éditeur (migrations 40 à 42)	`crates/pos-db/src/schema_invoicing.rs`
AccessPoint Peppol	`crates/pos-peppol/src/access_point.rs`
Commandes Peppol (enveloppe IPC : `src-tauri/src/ipc/invoice.rs`)	`crates/pos-engine/src/commands/invoice.rs`
Résolution du support + commandes ticket numérique	`crates/pos-engine/src/commands/delivery.rs`
Rendu e-mail (texte/HTML/QR)	`crates/pos-mail/src/render.rs`
Client Scaleway TEM	`crates/pos-mail/src/tem.rs`
File d’attente + réessais	`crates/pos-mail/src/outbox.rs`
Validation/masquage d’adresse	`crates/pos-mail/src/address.rs`
Requêtes `ticket_deliveries` / `original_prints`	`crates/pos-db/src/queries/{deliveries,prints}.rs`
Impression ESC/POS	`crates/pos-print/src/escpos.rs`
Layout ticket	`crates/pos-print/src/receipt.rs`
Transport imprimante	`crates/pos-print/src/transport.rs`